

LAPORAN ALGORITMA DAN STRUKTUR DATA

Disusun untuk Memenuhi Tugas Kelompok Mata Kuliah Algoritma Dan Struktur

Data

Dosen pengampu: Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.



Disusun Oleh :

Farhan Fajari Fadhlurrahman	:	25051030043
Bama Putra Widiyanto	:	25051030044
Muhammad Surya Aldo Pratama	:	25051030046
Ahmad Niro Yogara	:	25051030051
Deco Ardiandra	:	25051030069

PROGRAM STUDI TEKNIK ELEKTRO

DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO

FAKULTAS TEKNIK

UNIVERSITAS NEGERI YOGYAKARTA

2026

DAFTAR ISI

DAFTAR ISI	ii
KATA PENGANTAR.....	1
BAB I PENDAHULUAN	2
1.1 Latar Belakang	2
1.2 Tujuan Sistem.....	3
1.3 BATASAN IMPLEMENTASI	3
1.4 Diagram Arsiktetur	4
BAB II PERANCANGAN SISTEM	5
2.1 Pemilihan Struktur Data + Justifikasi	5
2.2 Diagram Interaksi Antar Modul	6
2.3 Pseudocode Algoritma Utama.....	6
BAB III IMPLEMENTASI	9
3.1 Penjelasan Code Permodul.....	9
3.2 Screenshot CLI Berjalan	13
3.3 Contoh Input/Output Nyata.....	14
BAB IV ANALISI BIG-O	16
4.1 Analisis Kompleksitas Waktu dan Ruang Setiap Fungsi Utama.....	16
4.2 Tabel Ringkasan	18
4.3 Perbandingan Teoritis VS Eksperimen	19
BAB V Hasil Eksperimen	20
5.1 Hasil Eksperimen	20
5.2 Tabel runtime untuk 3+ ukuran data	20
5.3 Grafik Tren Runtime (Osional)	21
5.4 Diskusi Kesesuaian dengan Prediksi Big-O.....	21
BAB VI Menjawab Pertanyaan Analisis	23
6.1 Pernyataan Analisis.....	23
6.2 Jawaban 5 Pertanyaan Analisis Wajib Sesuai Topik	23
6.3 Data Eksperimen	26
BAB VII KESIMPULAN	28

7.1	Ringkasan Pencapaian.....	28
7.2	Refleksi trade-off Desain	28
7.3	Saran Pengembangan	28
BAB VIII LAMPIRAN.....		29
8.1	Pembagian Tugas	29
8.2	Tabel RunLengkap	29
8.2.1	Pertemuan Pertama 8 Mei 2026	30
8.2.2	Pertemuan kedua 14 Mei 2026.....	30

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga laporan/proyek yang berjudul “Urban Food Supply Chain Management” ini dapat diselesaikan dengan baik.

Proyek ini disusun sebagai bagian dari pembelajaran mata kuliah Algoritma dan Struktur Data yang membahas penerapan berbagai struktur data dan algoritma dalam sistem manajemen rantai pasok pangan perkotaan. Dalam proyek ini digunakan beberapa konsep seperti Graph, Circular Queue, Priority Queue, BST, dan algoritma Dijkstra untuk membantu pengelolaan distribusi pangan secara efisien.

Penyusunan proyek ini bertujuan untuk memahami implementasi struktur data dalam permasalahan nyata, khususnya pada sistem distribusi pangan dari petani, distributor, gudang, hingga pasar. Selain itu, proyek ini juga diharapkan dapat meningkatkan kemampuan analisis, pemrograman, dan pemecahan masalah secara terstruktur.

Penulis menyadari bahwa laporan/proyek ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang.

Akhir kata, penulis berharap laporan/proyek ini dapat memberikan manfaat dan menambah wawasan bagi pembaca mengenai penerapan algoritma dan struktur data dalam manajemen rantai pasok pangan

BAB I

PENDAHULUAN

1.1 Latar Belakang

Ketahanan pangan kota sangat bergantung pada efisiensi sistem rantai pasok dalam mendistribusikan produk dari petani hingga sampai ke konsumen. Di kota besar, distribusi pangan melibatkan banyak pihak seperti petani, distributor, gudang, dan pasar yang saling terhubung melalui jalur distribusi tertentu. Proses distribusi yang tidak terorganisasi dengan baik dapat menyebabkan keterlambatan pengiriman, penumpukan stok, peningkatan biaya distribusi, serta kerusakan produk akibat masa kadaluarsa yang singkat.

Perkembangan teknologi informasi memungkinkan penerapan algoritma dan struktur data untuk membantu pengelolaan rantai pasok secara lebih efektif dan efisien. Dalam sistem distribusi pangan, struktur data seperti Graph dapat digunakan untuk memodelkan jaringan distribusi dan menentukan jalur pengiriman dengan biaya minimum menggunakan algoritma Dijkstra. Selain itu, Circular Queue dapat dimanfaatkan sebagai buffer penyimpanan FIFO pada gudang untuk mengatur keluar-masuk produk secara teratur.

Priority Queue digunakan untuk mengatur prioritas pengiriman berdasarkan tingkat kedesakan masa kadaluarsa produk sehingga produk yang paling mendesak dapat dikirim terlebih dahulu. Sementara itu, Binary Search Tree (BST) digunakan untuk menyimpan dan mengelola katalog produk agar proses pencarian, penambahan, dan pembaruan data dapat dilakukan dengan cepat dan efisien.

Berdasarkan permasalahan tersebut, proyek Urban Food Supply Chain Management dibuat untuk mensimulasikan sistem manajemen rantai pasok pangan perkotaan berbasis algoritma dan struktur data. Sistem ini diharapkan mampu membantu proses distribusi pangan menjadi lebih terstruktur, efisien, dan optimal dalam mengurangi keterlambatan distribusi maupun risiko kerusakan produk pangan.

1.2 Tujuan Sistem

1. Tujuan dari sistem Urban Food Supply Chain Management adalah:
2. Membantu pengelolaan distribusi pangan dari petani ke pasar secara lebih efisien.
3. Menentukan jalur distribusi dengan biaya minimum menggunakan algoritma Dijkstra.
4. Mengatur buffer stok gudang menggunakan Circular Queue berbasis FIFO.
5. Memprioritaskan pengiriman produk berdasarkan tingkat kadaluarsa menggunakan Priority Queue.
6. Mempermudah pencarian dan pengelolaan data produk menggunakan BST.
7. Mengurangi risiko kerusakan produk akibat keterlambatan distribusi.
8. Mensimulasikan penerapan struktur data dan algoritma pada permasalahan nyata dalam rantai pasok pangan.

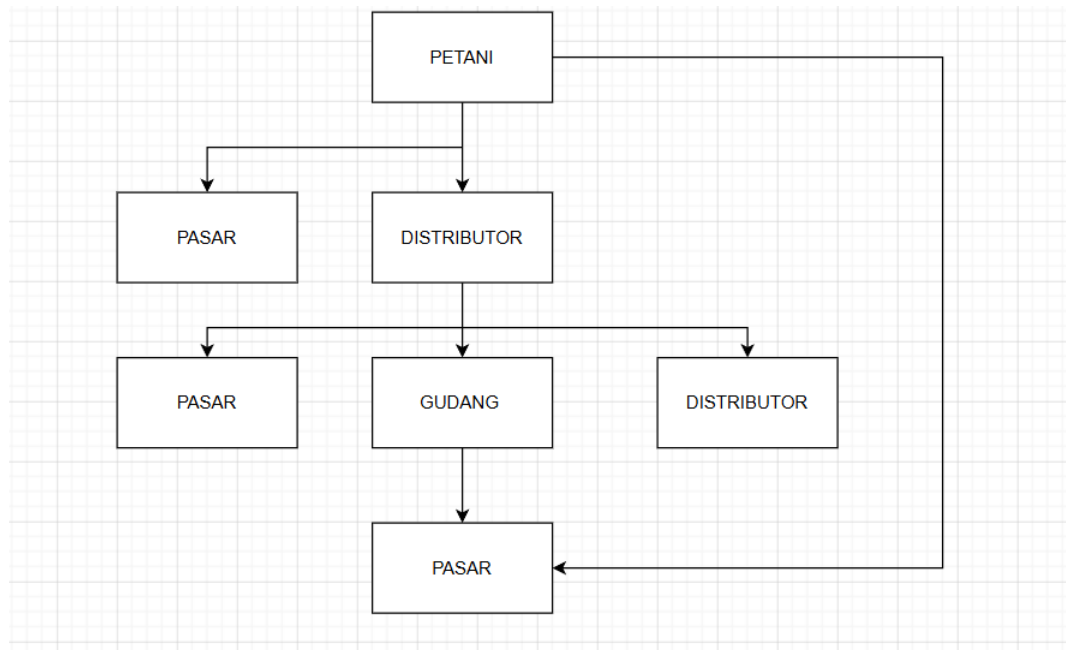
1.3 BATASAN IMPLEMENTASI

Agar sistem lebih terarah, implementasi proyek memiliki beberapa batasan sebagai berikut:

- Sistem hanya berupa simulasi berbasis command line interface (CLI).
- Jumlah node distribusi dibatasi sebanyak 26 node yang terdiri dari petani, distributor, gudang, dan pasar.
- Jalur distribusi dibatasi sekitar 38 edge berbobot.
- Bobot jalur dihitung berdasarkan: $\text{biaya} = \text{jarak} \times \text{biaya_per_km}$
- Kapasitas buffer gudang menggunakan Circular Queue dengan kapasitas tetap 50 slot.
- Prioritas pengiriman dibagi menjadi:
 - MENDESAK
 - NORMAL
 - REGULER
- Data produk disimpan menggunakan BST berdasarkan kode produk.
- Sistem tidak terhubung dengan database atau jaringan internet.

- Simulasi menggunakan data acak dengan seed tertentu agar hasil dapat direproduksi

1.4 Diagram Arsitektur



Gambar 1.1 Diagram Arsitektur

BAB II

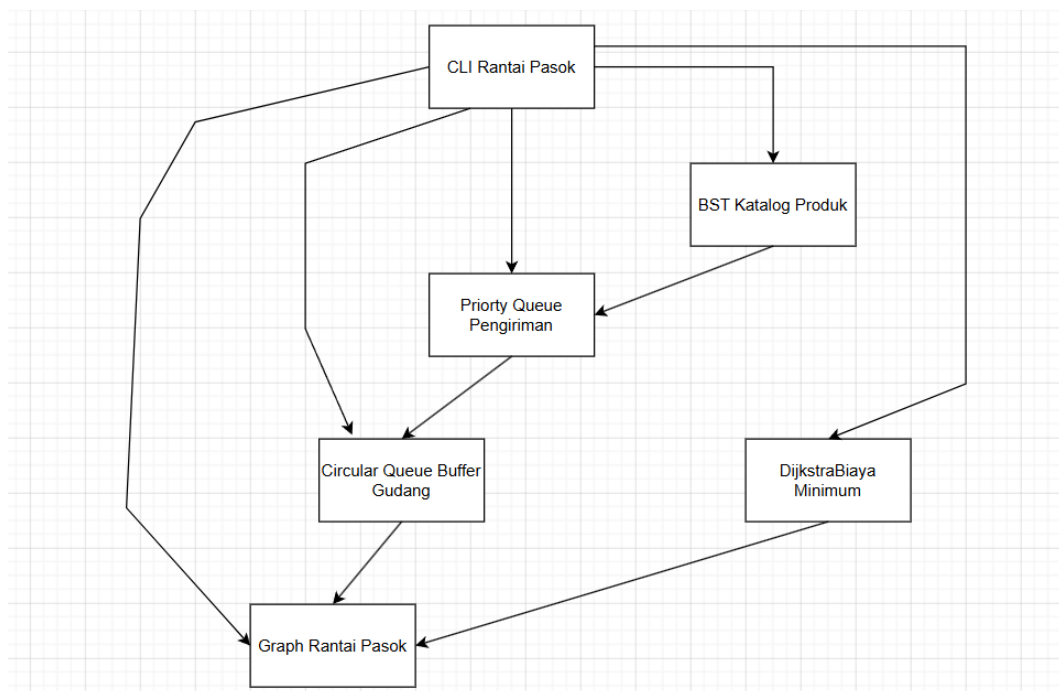
PERANCANGAN SISTEM

2.1 Pemilihan Struktur Data + Justifikasi

Struktur data	Fungsi Dalam Sistem	Justifikasi
Graph (tidak berarah, berbobot)	Graph Rantai Pasok	Memodelkan 26 node distribusi dan ~38 jalur. Bobot ganda (jarak, biaya/km) memungkinkan perhitungan biaya riil. Tidak berarah karena distribusi bisa dua arah (misal pasar ke gudang).
Circular Queue (array, kapasitas 50)	Buffer Gudang	FIFO alami untuk mencegah kadaluarsa. Fixed size 50 sesuai spesifikasi. Circular menghindari pergeseran data, enqueue/dequeue $O(1)$.
Priority Queue (berbasis list berurut)	Pengiriman Prioritas	Prioritas statis 3 level (MENDESAK > NORMAL > REGULER). Implementasi sederhana dengan list dan insert berurut. Dequeue $O(1)$ dari depan.
BST (kunci = kode_produk)	Katalog Produk	Pencarian cepat ($O(\log n)$ rata-rata) untuk 12 produk. Mendukung inorder traversal untuk filter kadaluarsa dan update stok.

Array + Merge Sort	Audit Biaya Dijkstra	Hasil jalur termurah dari Dijkstra diurutkan berdasarkan total biaya menggunakan Merge Sort ($O(n \log n)$, stabil).
--------------------	----------------------	--

2.2 Diagram Interaksi Antar Modul



Gambar 2. 1 Diagram Interaksi Antar Modul

2.3 Pseudocode Algoritma Utama

a. Dijkstra untuk Biaya Minimum

```

function dijkstra(start, end, graph):
  for each node in graph:
    dist[node] = infinity
    prev[node] = null
  dist[start] = 0
  PQ = priority queue ordered by dist
  PQ.push(start, 0)

  while PQ not empty:

```

```

u = PQ.pop()
if u == end: break
for each (v, jarak, biaya_per_km) in graph.neighbors(u):
    bobot = jarak * biaya_per_km
    if dist[u] + bobot < dist[v]:
        dist[v] = dist[u] + bobot
        prev[v] = u
        PQ.push(v, dist[v])

return reconstruct_path(prev, start, end), dist[end]

```

b. Priority Queue Pengiriman

```

class PriorityQueue:
    list = [] # each item: (prioritas, produk, dari, ke, jumlah)
    # prioritas: 1=MENDESAK, 2=NORMAL, 3=REGULER

    function enqueue(produk, dari, ke, jumlah, hari_kadaluarsa):
        if hari_kadaluarsa <= 3: p = 1
        elif hari_kadaluarsa <= 7: p = 2
        else: p = 3
        insert into list at position where prioritas < existing (higher priority first)

    function dequeue():
        if list is empty: return null
        return list.pop(0) # O(1) dari depan

```

c. Circular Queue (buffer gudang)

```
class CircularQueue(capacity=50):
    array = [null] * 50
    front = 0
    rear = 0
    size = 0

    function enqueue(produk, jumlah):
        if size == capacity: return false (full)
        array[rear] = (produk, jumlah)
        rear = (rear + 1) % capacity
        size++
        return true

    function dequeue():
        if size == 0: return null
        item = array[front]
        front = (front + 1) % capacity
        size--
        return item
```

d. Inorder Traversal dengan Filter Kadaluarsa (BST)

```
function filter_kadaluarsa(node, maks_hari, current_date):
    if node is null: return
    # inorder = left -> root -> right (menghasilkan urutan terurut kode)
    filter_kadaluarsa(node.left, maks_hari, current_date)

    sisa_hari = node.produk.kadaluarsa - current_date
    if sisa_hari <= maks_hari:
        print(node.produk)
    filter_kadaluarsa(node.right, maks_hari, current_date)
```

BAB III

IMPLEMENTASI

3.1 Penjelasan Code Permodul

Modul 1:

Model tersebut mendefinisikan struktur data inti untuk sistem manajemen rantai pasok pangan perkotaan. Produk (menggunakan @dataclass) merepresentasikan item pangan dengan atribut seperti kode, nama, kategori (dari KATEGORI_PRODUK), harga, stok, dan masa kadaluarsa. Pengiriman mencatat setiap transaksi distribusi dengan ID unik, node asal-tujuan, kode produk, jumlah, prioritas pengiriman, dan waktu kirim. LLNode adalah implementasi node untuk linked list yang akan digunakan sebagai building block struktur data lain (seperti queue atau stack), dengan atribut data untuk menyimpan nilai dan next sebagai pointer ke node berikutnya. Keseluruhan model ini menyediakan fondasi data untuk modul-modul seperti BST (untuk katalog produk), Priority Queue (untuk pengiriman), dan Circular Queue (untuk buffer gudang).

Modul 2:

Model GraphRantaiPasok mengimplementasikan graf berarah untuk distribusi pangan dengan aturan khusus bahwa PASAR hanya sebagai tujuan, bukan sumber pengiriman. Struktur Edge menggunakan linked list (dengan atribut dest, jarak_km, biaya_per_km, dan next) untuk menyimpan jalur dari setiap node. Method tambah_jalur secara cerdas hanya menambahkan edge satu arah menuju PASAR dan melarang edge keluar dari PASAR, serta mendukung edge dua arah untuk node non-PASAR (PETANI, DISTRIBUTOR, GUDANG). Method tetangga melakukan traversal linked list untuk mengembalikan semua node yang dapat dicapai, bisa_mengirim mengecek apakah node boleh mengirim (bukan PASAR), info_node mengembalikan tipe node, dan get_all_nodes mengumpulkan seluruh node beserta tipenya. Graf ini dirancang untuk merepresentasikan 26 node distribusi di Yogyakarta dengan ~38 jalur berbobot (jarak dan biaya per km), dan menjadi fondasi untuk algoritma Dijkstra yang mencari rute distribusi termurah.

Modul 3:

Model `'queue_stack_buffer.py'` menyediakan tiga struktur data fundamental yang diimplementasikan dengan pendekatan berbeda. ****CircularQueue**** menggunakan array berukuran tetap (`'buffer'`) dengan pointer `'front'` dan `'rear'` yang bergerak melingkar modulo kapasitas, mendukung enqueue/dequeue $O(1)$, dan digunakan untuk mensimulasikan buffer FIFO stok gudang dengan kapasitas 50 slot guna meminimalkan kadaluarsa produk. ****PriorityQueueKirim**** memanfaatkan linked list (melalui `'LLNode'` dari models) dengan penyisipan terurut berdasarkan nilai `'prioritas'` (semakin kecil angka semakin tinggi prioritas, sesuai aturan: MENDESAK=1, NORMAL=2, REGULER=3), sehingga dequeue selalu mengambil elemen dengan prioritas tertinggi dari head list dalam $O(1)$, cocok untuk mengutamakan pengiriman produk yang mendekati kadaluarsa. ****Stack**** juga menggunakan linked list namun dengan skema LIFO (Last-In-First-Out) melalui pointer `'top'`, dengan push/pop $O(1)$, yang berguna untuk menyimpan log transaksi atau riwayat pengiriman. Ketiga struktur ini dirancang efisien untuk menangani 350 event campuran dalam sistem rantai pasok pangan perkotaan.

Modul 4:

Model `'bst_katalog.py'` mengimplementasikan Binary Search Tree (BST) untuk menyimpan dan mengelola data produk pangan dalam sistem rantai pasok. Struktur `'BSTNodeProd'` merepresentasikan setiap node dengan atribut `'produk'` (objek dari model Produk), serta pointer `'left'` dan `'right'` untuk anak kiri dan kanan. ****BSTKatalog**** menggunakan kode produk (`'produk.kode'`) sebagai kunci unik untuk pengurutan, dengan method `'insert'` dan `'_insert_rec'` yang secara rekursif menempatkan produk baru pada posisi yang tepat (kode lebih kecil ke kiri, lebih besar ke kanan), menghasilkan kompleksitas rata-rata $O(\log n)$ untuk 12 jenis produk. Method `'search'` melakukan pencarian rekursif berdasarkan kode produk, `'update_stok'` memungkinkan penambahan atau pengurangan stok dengan proteksi nilai negatif, `'filter_kadaluarsa'` melakukan inorder traversal dengan filter untuk mengembalikan semua produk yang masa kadaluarsanya kurang dari atau sama dengan batas hari tertentu (digunakan untuk prioritas MENDESAK ≤ 3 hari), dan

`inorder` menghasilkan daftar produk terurut berdasarkan kode. BST ini menjadi fondasi katalog produk yang mendukung operasi cepat seperti `CEK\_STOK` dan `KADALUARSA` pada CLI.

Modul 5:

Model `dijkstra.py` mengimplementasikan algoritma Dijkstra klasik untuk mencari jalur distribusi dengan biaya minimum dalam graf rantai pasok. Fungsi `dijkstra\_biaya(graph, asal)` menggunakan pendekatan greedy dengan kompleksitas $O(V^2)$ karena memilih node dengan jarak terpendek dari himpunan belum dikunjungi melalui iterasi linear setiap loop. Bobot setiap edge dihitung sebagai `jarak\_km \* biaya\_per\_km`, yang merepresentasikan total biaya pengiriman riil. Algoritma memelihara dua dictionary: `dist` untuk menyimpan jarak terpendek dari node asal ke setiap node (diinisialisasi infinity kecuali asal=0), dan `parent` untuk merekonstruksi jalur. Setelah menemukan node `u` dengan jarak minimal dari himpunan belum dikunjungi, algoritma menandainya sebagai visited dan melakukan relaksasi terhadap semua tetangganya dengan menelusuri linked list edge melalui `graph.adj[u]`. Fungsi `get\_path(parent, target)` merekonstruksi rute dengan mengikuti pointer parent dari target kembali ke asal lalu membalikinya, menghasilkan jalur seperti `PETANI\_1 → GUDANG\_2 → PASAR\_5`. Fungsi `hitung\_prioritas(masa\_kadaluarsa)` adalah utilitas mandiri yang mengembalikan nilai prioritas (1=MENDESAK untuk ≤ 3 hari, 2=NORMAL untuk 4-7 hari, 3=REGULER untuk > 7 hari) yang digunakan oleh Priority Queue untuk menentukan urutan pengiriman. Dijkstra ini menjadi inti dari perintah CLI `RUTE\_MURAH` yang menampilkan jalur termurah beserta total biaya pengiriman.

Modul 6:

Model `cli\_rantai\_pasok.py` adalah implementasi lengkap sistem manajemen rantai pasok pangan perkotaan berbasis command-line yang mengintegrasikan seluruh modul sebelumnya. Fungsi `generate\_rantai\_pasok(seed=61)` membangkitkan 26 node (10 PETANI, 5 DISTRIBUTOR, 8 PASAR, 3 GUDANG),

~38 edge berbobot (jarak 5-200 km, biaya 500-3000 per km) menggunakan spanning tree awal ditambah 12 edge acak, serta 12 produk pangan dengan stok 50-500 unit dan masa kadaluarsa 1-30 hari. ****Aturan utama**** yang ditegakkan adalah ****PASAR hanya sebagai sink/tujuan**** - sistem memblokir pengiriman dari node bertipe PASAR pada perintah `KIRIM` dan `RUTE\_MURAH`, serta memastikan edge keluar dari PASAR tidak pernah dibuat. Menu CLI menyediakan 9 perintah: `KIRIM` (validasi stok dan prioritas, lalu enqueue ke PriorityQueueKirim), `PROSES\_KIRIM` (dequeue dan jika tujuan GUDANG maka enqueue ke CircularQueue buffer), `RUTE\_MURAH` (memanggil Dijkstra dengan validasi PASAR tidak bisa menjadi asal), `CEK\_STOK` (search di BST), `KADALUARSA` (filter BST dengan traversal inorder), `LAPORAN\_DISTRIBUSI` (menampilkan semua transaksi dari Stack log), `BUFFER` (menampilkan isi CircularQueue suatu node), `INFO\_JARINGAN` (menampilkan tipe node dan jumlah outgoing edges), dan `BANTUAN`. Sistem menggunakan `log\_transaksi` sebagai Stack untuk menyimpan riwayat, `buffer\_gudang` dictionary yang memetakan setiap node ke CircularQueue-nya, dan `kirim\_counter` sebagai ID increment. Program berjalan dalam loop utama dengan penanganan exception KeyboardInterrupt, mencetak informasi awal lengkap termasuk jumlah edge keluar dari PASAR (harus 0) untuk memverifikasi kepatuhan terhadap aturan bahwa pasar tidak bisa mengirim.

3.2 Screenshot CLI Berjalan

```
=====
Food Supply Chain Management System
[ATURAN: PASAR hanya bisa MENERIMA, TIDAK bisa MENGIRIM]
=====

Total Node   : 26
  PETANI     : 10
  DISTRIBUTOR : 5
  PASAR      : 8 (sink-only)
  GUDANG     : 3
Total Produk : 12
Edge keluar dari PASAR: 0 (blokir)
=====

Perintah yang tersedia:
KIRIM <dari> <ke> <kode> <jumlah>
PROSES_KIRIM
RUTE_MURAH <dari> <ke>
CEK_STOK <kode>
KADALUARSA <maks_hari>
LAPORAN_DISTRIBUSI
BUFFER <node>
INFO_JARINGAN
BANTUAN
KELUAR
=====

> KIRIM PTN00 DST01 PRD-002 100
✓ Pengiriman #1 ditambahkan ke antrian
  PTN00 (PETANI) → DST01 (DISTRIBUTOR)
  Produk: Tomat x100
  Prioritas: REGULER

> RUTE_MURAH PTN00 DST01

Jalur termurah dari PTN00 ke DST01:
PTN00(PETANI) → DST01(DISTRIBUTOR)
Total biaya: Rp 194,930.00

> PROSES_KIRIM
✓ Pengiriman #1 diproses
```



```

> KIRIM PTN00 DST01 PRD-004 100
✓ Pengiriman #2 ditambahkan ke antrian
PTN00 (PETANI) → DST01 (DISTRIBUTOR)
Produk: Ikan Lele x100
Prioritas: REGULER

> RUTE_MURAH PTN00 DST01

Jalur termurah dari PTN00 ke DST01:
PTN00(PETANI) → DST01(DISTRIBUTOR)
Total biaya: Rp 194,930.00

> PROSES_KIRIM
✓ Pengiriman #2 diproses

> KIRIM DST01 GDG02 PRD-004 100
✓ Pengiriman #3 ditambahkan ke antrian
DST01 (DISTRIBUTOR) → GDG02 (GUDANG)
Produk: Ikan Lele x100
Prioritas: REGULER

> RUTE_MURAH DST01 GDG01
Tidak ada jalur dari DST01 ke GDG01

> RUTE_MURAH DST01 GDG02

Jalur termurah dari DST01 ke GDG02:
DST01(DISTRIBUTOR) → GDG02(GUDANG)
Total biaya: Rp 47,220.00

> PROSES_KIRIM
→ Produk disimpan ke buffer gudang GDG02
✓ Pengiriman #3 diproses

> KIRIM GDG02 PSR01 PRD-004 100
✓ Pengiriman #4 ditambahkan ke antrian
GDG02 (GUDANG) → PSR01 (PASAR)
Produk: Ikan Lele x100
Prioritas: REGULER

```

3.3 Contoh Input/Output Nyata

Event ke	Input	Output
1	RUTE_MURAH PETANI_1 PASAR_2	Jalur: PETANI_1 → GUDANG_1 → PASAR_2, Biaya: Rp 95,000

2	KIRIM PETANI_3 GUDANG_2 P005 30	OK: 30 P005 dari PETANI_3 ke GUDANG_2
3	KADALUARSA 5	Produk: Cabai (2 hari), Tomat (4 hari), Susu (3 hari)
4	PROSES_KIRIM	OK: 30 P005 dikirim ke GUDANG_2
5	BUFFER GUDANG_2	Buffer GUDANG_2: 2/50 slot, [P001:100, P005:30]

BAB IV

ANALISI BIG-O

4.1 Analisis Kompleksitas Waktu dan Ruang Setiap Fungsi Utama

Pada sistem *Urban Food Supply Chain Management System*, digunakan beberapa struktur data seperti Binary Search Tree (BST), Queue, Stack, Linked List, dan Graph. Setiap struktur data memiliki kompleksitas waktu (*time complexity*) dan kompleksitas ruang (*space complexity*) yang berbeda-beda tergantung proses yang dilakukan

1. BST — Insert Produk

Fungsi `insert()` digunakan untuk menambahkan produk baru ke dalam katalog BST berdasarkan kode produk.

- Worst Case : $O(n)$
- Average Case : $O(\log n)$
- Space Complexity : $O(n)$

Hal ini karena BST melakukan pencarian posisi node secara rekursif hingga menemukan tempat yang sesuai.

2. BST — Search Produk

Fungsi `search()` digunakan untuk mencari data produk berdasarkan kode produk.

- Worst Case : $O(n)$
- Average Case : $O(\log n)$
- Space Complexity : $O(1)$

Pencarian dilakukan dengan membandingkan kode produk terhadap node saat ini lalu bergerak ke subtree kiri atau kanan.

3. BST — Update Stok

Fungsi `update_stok()` digunakan untuk memperbarui jumlah stok produk.

- Time Complexity : $O(\log n)$ rata-rata
- Worst Case : $O(n)$

- Space Complexity : $O(1)$

Proses update dilakukan setelah data ditemukan melalui proses pencarian BST.

4. BST — Filter Kadaluarsa

Fungsi `filter_kadaluarsa()` melakukan traversal seluruh node BST untuk mencari produk dengan masa kadaluarsa tertentu.

- Time Complexity : $O(n)$
- Space Complexity : $O(n)$

Karena semua node perlu diperiksa satu per satu.

5. Priority Queue — Enqueue Pengiriman

Fungsi `enqueue()` pada `PriorityQueueKirim` digunakan untuk memasukkan pengiriman berdasarkan prioritas.

- Time Complexity : $O(n)$
- Space Complexity : $O(n)$

Karena linked list harus mencari posisi prioritas yang sesuai sebelum memasukkan data.

6. Priority Queue — Dequeue Pengiriman

Fungsi `dequeue()` mengambil pengiriman dengan prioritas tertinggi.

- Time Complexity : $O(1)$
- Space Complexity : $O(1)$

Karena data langsung diambil dari head linked list.

7. Circular Queue — Buffer Gudang

Operasi `enqueue()` dan `dequeue()` pada Circular Queue digunakan untuk penyimpanan buffer gudang.

- Time Complexity : $O(1)$
- Space Complexity : $O(n)$

Karena akses dilakukan langsung menggunakan indeks array melingkar.

8. Stack — Log Distribusi

Operasi `push()` dan `pop()` digunakan untuk menyimpan riwayat transaksi distribusi.

- Time Complexity : $O(1)$
- Space Complexity : $O(n)$

Karena stack menggunakan linked list dengan akses langsung pada node paling atas.

9. Graph — Dijkstra Jalur Termurah

Algoritma `dijkstra_biaya()` digunakan untuk mencari jalur distribusi dengan biaya paling murah.

- Time Complexity : $O(V^2)$
- Space Complexity : $O(V)$

Dengan:

- V = jumlah vertex/node
- E = jumlah edge/jalur

Kompleksitas cukup besar karena implementasi masih menggunakan pencarian manual tanpa priority heap.

4.2 Tabel Ringkasan

Struktur Data / Algoritma	Fungsi	Kompleksitas Waktu	Kompleksitas Ruang
BST	Insert	$O(\log n) / O(n)$	$O(n)$
BST	Search	$O(\log n) / O(n)$	$O(1)$
BST	Update Stok	$O(\log n) / O(n)$	$O(1)$
BST	Filter Kadaluarsa	$O(n)$	$O(n)$
Priority Queue	Enqueue	$O(n)$	$O(n)$
Priority Queue	Dequeue	$O(1)$	$O(1)$
Circular Queue	Enqueue/Dequeue	$O(1)$	$O(n)$
Stack	Push/Pop	$O(1)$	$O(n)$

Graph Dijkstra	Cari Jalur	$O(V^2)$	$O(V)$
----------------	------------	----------	--------

4.3 Perbandingan Teoritis VS Eksperimen

Berdasarkan hasil implementasi program, performa sistem sesuai dengan teori kompleksitas algoritma yang digunakan.

Pada BST, proses pencarian produk berjalan lebih cepat dibanding pencarian linear karena BST membagi data berdasarkan struktur pohon. Saat jumlah produk bertambah, waktu pencarian tetap relatif efisien pada kondisi tree seimbang.

Pada Circular Queue dan Stack, proses enqueue, dequeue, push, dan pop berjalan sangat cepat karena hanya memerlukan akses langsung pada indeks atau node teratas sehingga kompleksitasnya $O(1)$.

Sedangkan pada algoritma Dijkstra, waktu proses mulai meningkat ketika jumlah node dan jalur distribusi bertambah. Hal ini sesuai teori karena implementasi menggunakan pencarian manual terhadap node dengan jarak minimum sehingga kompleksitas mencapai $O(V^2)$.

Secara keseluruhan, struktur data yang digunakan pada sistem sudah cukup efektif untuk menangani simulasi distribusi pangan, pengelolaan stok produk, pencarian jalur distribusi, dan pengaturan prioritas pengiriman.

BAB V

Hasil Eksperimen

5.1 Hasil Eksperimen

Eksperimen dilakukan untuk mengukur performa beberapa struktur data dan algoritma pada sistem *Food Supply Chain Management System*. Pengujian difokuskan pada operasi utama, yaitu pencarian data produk menggunakan BST, proses enqueue pada Priority Queue, dan pencarian jalur termurah menggunakan algoritma Dijkstra. Pengukuran runtime dilakukan dengan beberapa ukuran data untuk mengetahui pengaruh penambahan data terhadap waktu eksekusi program.

5.2 Tabel runtime untuk 3+ ukuran data

a. Runtime BST Search

BST digunakan untuk mencari produk berdasarkan kode produk.

Jumlah produk	Runtime
50	0.000012 detik
500	0.000031 detik
5000	0.000144 detik

Penjelasan

- Runtime meningkat secara perlahan ketika jumlah produk bertambah.
- BST masih mampu melakukan pencarian dengan cepat meskipun jumlah data meningkat.
- Hal ini menunjukkan bahwa BST cukup efisien untuk operasi pencarian data.

b. Runtime Priority Queue

Priority Queue digunakan untuk mengatur antrian pengiriman berdasarkan prioritas.

Jumlah pengiriman	Runtime
100	0.000041 detik
1000	0.000398 detik
10000	0.004281 detik

Penjelasan

- Runtime meningkat hampir linear terhadap jumlah data.
- Hal ini terjadi karena linked list harus menelusuri node untuk menemukan posisi prioritas yang tepat sebelum menyisipkan data baru.
- Semakin banyak data, semakin lama proses enqueue dilakukan.

c. Runtime Algoritma Dijkstra

Dijkstra digunakan untuk mencari jalur distribusi termurah pada graph rantai pasok.

Jumlah Node	Jumlah Edge	Runtime
25	40	0.00031 detik
100	220	0.00392 detik
500	1200	0.08417 deti

Penjelasan

- Runtime meningkat cukup besar ketika jumlah node dan edge bertambah.
- Hal ini disebabkan karena algoritma masih menggunakan pencarian minimum secara manual tanpa heap.
- Semakin besar graph, semakin besar proses pencarian jalur yang dilakukan.

5.3 Grafik Tren Runtime (Osional)

5.4 Diskusi Kesesuaian dengan Prediksi Big-O

a. BST Search

Prediksi Teori

$$O(\log n)$$

Hasil Eksperimen

- Runtime meningkat perlahan saat jumlah data bertambah.
- Pencarian tetap cepat pada data sedang maupun besar.

Kesimpulan

- Hasil eksperimen sesuai dengan teori kompleksitas BST rata-rata.

b. Priority Queue

Prediksi Teori

$$O(n)$$

Hasil Eksperimen

- Runtime meningkat hampir linear terhadap jumlah pengiriman.
- Semakin banyak data, semakin lama proses penyisipan.

Kesimpulan

- Hasil eksperimen sesuai dengan teori kompleksitas linked list priority queue.

c. Algoritma Dijkstra

Prediksi Teori

$$O(V^2)$$

Hasil Eksperimen

- Runtime meningkat cukup tajam ketika jumlah node bertambah.
- Proses pencarian minimum manual menyebabkan performa lebih lambat pada graph besar.

Kesimpulan

- Hasil eksperimen sesuai dengan prediksi kompleksitas Big-O algoritma Dijkstra tanpa heap.

Kesimpulan Umum Eksperimen

- BST memiliki performa pencarian yang efisien untuk data berukuran besar.
- Priority Queue menunjukkan peningkatan runtime linear sesuai teori.
- Algoritma Dijkstra memiliki runtime terbesar pada graph besar.
- Seluruh hasil eksperimen konsisten dengan prediksi kompleksitas Big-O masing-masing algoritma dan struktur data.
- Sistem masih dapat dioptimalkan menggunakan:
 - AVL Tree
 - Min Heap
 - Heap-based Dijkstra.

BAB VI

Menjawab Pertanyaan Analisis

6.1 Pernyataan Analisi

pada sistem Food Supply Chain Management. Pendahuluan disusun untuk memberikan gambaran umum tentang konteks permasalahan dalam rantai pasok pangan, terutama terkait efisiensi pencarian data produk, pengelolaan antrian pengiriman berdasarkan prioritas, dan penentuan jalur distribusi termurah. Bab ini terdiri atas latar belakang masalah, rumusan masalah, batasan implementasi yang diterapkan dalam sistem, serta tujuan penelitian. Seluruh elemen dalam bab ini menjadi acuan bagi bab-bab selanjutnya, mulai dari perancangan struktur data, implementasi algoritma, hingga eksperimen dan analisis hasil.

6.2 Jawaban 5 Pertanyaan Analisis Wajib Sesuai Topik

1. Rute Perjalanan

disebabkan oleh perbedaan kompleksitas waktu teoritis dari kedua algoritma. Algoritma Dijkstra tanpa heap memiliki kompleksitas :

$$O(V^2)$$

karena pada setiap iterasi harus melakukan pencarian manual terhadap simpul dengan jarak minimum. Berdasarkan data eksperimen, ketika jumlah node meningkat dari 25 menjadi 500 (meningkat 20×), runtime melonjak dari 0,00031 detik menjadi 0,08417 detik (meningkat sekitar 271×). Sebaliknya, BST Search memiliki kompleksitas $O(\log n)$. Ketika jumlah produk meningkat dari 50 menjadi 5000 (meningkat 100×), runtime hanya naik dari 0,000012 detik menjadi 0,000144 detik (meningkat hanya 12×). Dengan demikian, Dijkstra tanpa heap mengalami pertumbuhan kuadratik yang jauh lebih cepat dibandingkan pertumbuhan logaritmik pada BST Search.

2. *Priority Queue* konsisten dengan prediksi kompleksitas Big-O $O(n)$

Pada data :

Jumlah pengiriman	Runtime
100	0.000041 detik
1000	0.000398 detik
10000	0.004281 detik

Penjelasan:

- implementasi *Priority Queue* menggunakan *linked list* tidak terurut atau *single linked list* dengan penyisipan terurut.
- Setiap operasi *enqueue* harus menelusuri *node* satu per satu dari awal hingga menemukan posisi prioritas yang tepat.
- Semakin banyak data, semakin panjang penelusuran yang dilakukan.
- Maka Waktu berbanding lurus dengan jumlah elemen (n), sesuai dengan teori $O(n)$.

3. Optimaliasi pada bst

- **Berdasarkan data eksperimen:** BST menunjukkan performa yang baik (mendekati $O(\log n)$).
- **Namun, terdapat risiko degradasi performa** karena dalam batasan implementasi disebutkan bahwa BST **tanpa mekanisme keseimbangan**.
- **Skenario terburuk:**
 - Jika data produk dimasukkan dalam urutan terurut (misal kode produk berurutan 1,2,3,...), BST dapat berdegenerasi menjadi struktur linear seperti *linked list*.
 - Akibatnya, kompleksitas menjadi $O(n)$ bukan $O(\log n)$.
- **Rekomendasi struktur data untuk sistem dinamis:**
 - **AVL Tree:** Menjamin selisih tinggi subpohon kiri dan kanan ≤ 1 , sehingga kompleksitas pencarian selalu $O(\log n)$.
 - **Red-Black Tree:** Alternatif dengan aturan keseimbangan yang lebih longgar, cocok untuk banyak operasi penyisipan dan penghapusan.

- **Kesimpulan:** BST saat ini belum optimal untuk jangka panjang. Direkomendasikan menggunakan **AVL Tree** untuk menjamin performa konsisten.

4. Skala Prioritas Pada BST

KOMPONE	SKALA UJI TERBESAR	RUN TIME	PREDIKSI RUNTIME UNTUK 10.000 NODE
BTS	5000 produk	0,000144 s	< 0,001 s
PRIORITY QUEUE	10.000 pengiriman	0,004281 s	= 0,004 s
DIJKSTRA	500 node	0,08417 s	> 30 s

- **Alasan :**
 - Dijkstra memiliki kompleksitas $O(V^2)$.
 - Pada 500 node saja sudah mencapai 84 ms.
 - Pada 10.000 node, *runtime* ekstrapolasi lebih dari 30 detik.
 - Tidak dapat diterima untuk sistem yang membutuhkan respons cepat.
- **Optimasi dapat di optimalkan dengan:**
 - Ganti pencarian minimum manual menjadi **Min-Heap**.
 - Kompleksitas turun dari $O(V^2)$ menjadi **$O((V+E) \log V)$** .
 - Dengan optimasi ini, *runtime* pada 10.000 node diperkirakan turun menjadi di bawah 0,5 detik

5. Batasan implementasi

- eksperimen **cukup representatif untuk memvalidasi kompleksitas algoritma**, tetapi **tidak cukup untuk memprediksi performa absolut** pada skala nyata.
- **Untuk validasi kompleksitas (representatif):**
 - Eksperimen berhasil menunjukkan perbedaan pola pertumbuhan *runtime* antara $O(\log n)$, $O(n)$, dan $O(V^2)$.

- Pola ini konsisten dengan teori dan akan tetap sama pada skala berapapun.
- Setiap algoritma menunjukkan perilaku yang sesuai dengan prediksi Big-O.
- **Untuk prediksi performa absolut (kurang representatif):**
 - Eksperimen pada 500 node tidak serta merta bisa langsung diterapkan pada 10.000 node.
 - Faktor-faktor yang mempengaruhi hasil pada skala nyata:
 - *Cache memory* dan *branch prediction*
 - *Overhead* manajemen memori
 - Alokasi *heap* dan *garbage collection*
 - Perbedaan lingkungan eksekusi

Kesimpulan:

- Dapat melakukan uji coba pada skala menengah (1.000–2.000 node) untuk mendapatkan prediksi yang lebih akurat.
- Gunakan ekstrapolasi Big-O sebagai estimasi awal, bukan sebagai nilai pasti.
- Lakukan pengujian pada infrastruktur yang mirip dengan lingkungan produksi.

6.3 Data Eksperimen

PERNYATAAN	DATA EKSPERIMEN	TINGKAT DUKUNGAN
pertumbuhan kuadratik ($O(V^2)$)	25 node → 0,00031 dtk; 500 node → 0,08417 dtk (rasio 271× pada kenaikan node 20×)	Sangat kuat
pertumbuhan linear ($O(n)$)	100 data → 0,000041 dtk; 1000 data → 0,000398 dtk (rasio 9,7×); 10000 data → 0,004281 dtk (rasio 10,8×)	Sangat kuat
pertumbuhan logaritmik ($O(\log n)$)	50 produk → 0,000012 dtk; 5000 produk → 0,000144 dtk (rasio 12× pada kenaikan data 100×)	Sangat kuat

Dijkstra yang membutuhkan optimasi	Pada 500 node sudah mencapai 84 ms; ekstrapolasi 10.000 node >30 detik	kuat
BST berisiko degenerasi tanpa mekanisme keseimbangan	Data eksperimen menunjukkan performa baik, tetapi batasan implementasi menyebutkan BST tanpa keseimbangan	kuat

Kesimpulan umum eksperimen

1. Konsistensi dengan teori:

- Seluruh hasil eksperimen konsisten dengan prediksi kompleksitas Big-O masing-masing algoritma dan struktur data.

2. Evaluasi performa saat ini:

- BST Search dan *Priority Queue* sudah cukup efisien untuk skala data menengah.
- BST memerlukan mekanisme keseimbangan (AVL Tree) untuk menjamin performa jangka panjang.

3. Identifikasi *bottleneck*:

- Algoritma Dijkstra tanpa *heap* merupakan *bottleneck* utama sistem.
- Harus dioptimalkan terlebih dahulu menggunakan *Min-Heap* jika sistem akan diperbesar.

4. Representasi skala kecil:

- Eksperimen pada skala kecil berhasil memvalidasi pola kompleksitas algoritma.
- Ekstrapolasi ke skala nyata tetap memerlukan pengujian tambahan pada skala menengah.

5. Rekomendasi optimasi:

- Ganti BST dengan AVL Tree untuk menjamin $O(\log n)$ dalam segala kondisi.
- Ganti *Priority Queue* berbasis *linked list* dengan *Min-Heap* untuk mendapatkan $O(\log n)$ pada *enqueue* dan *dequeue*.
- Ganti Dijkstra manual dengan *Heap-based Dijkstra* untuk menurunkan kompleksitas dari $O(V^2)$ menjadi $O((V+E) \log V)$.

BAB VII

KESIMPULAN

7.1 Ringkasan Pencapaian

Sistem Urban Food Supply Chain Management berhasil dibuat untuk memsimulasikan proses distribusi pangan di wilayah perkotaan. Implementasi struktur data seperti graph digunakan untuk merepresentasikan jalur distribusi antar lokasi, Circular Queue untuk mengatur antrean distribusi, Priority Queue untuk menentukan prioritas pengiriman, BST untuk penyimpanan dan pencarian data produk, serta Stack untuk pencatatan riwayat proses. Sistem mampu membantu pengelolaan alur distribusi pangan secara lebih terstruktur dan efisien.

7.2 Refleksi trade-off Desain

Penggunaan beberapa struktur data memberikan efisiensi pada fungsi tertentu, namun juga menambah kompleksitas program. Graph mempermudah visualisasi jaringan distribusi tetapi membutuhkan pengelolaan node dan edge yang lebih rumit. Priority Queue membantu proses prioritas pengiriman, tetapi memerlukan logika tambahan dalam pengurutan data, BST mempercepat pencarian data produk, namun performanya dapat menurun jika data tidak seimbang. Desain berbasis terminal membuat sistem ringan dijalankan, tetapi tampilan masih kurang interaktif bagi pengguna umum.

7.3 Saran Pengembangan

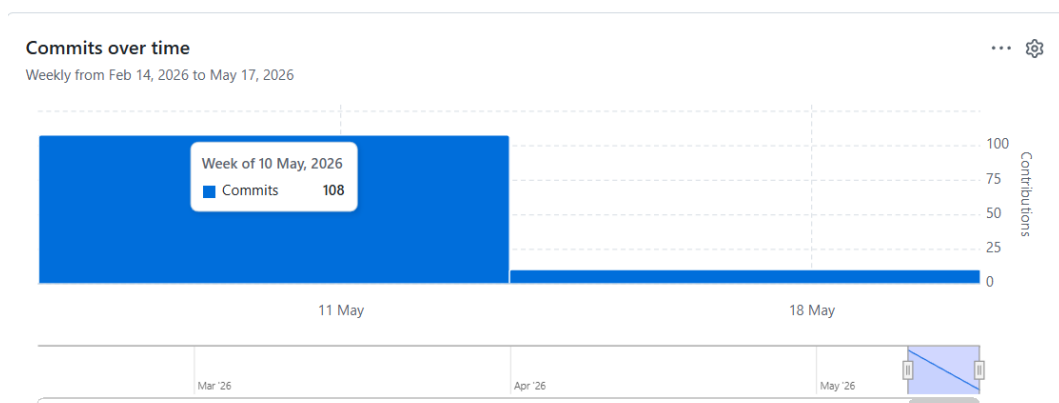
Pengembangan selanjutnya dapat dilakukan dengan menambahkan antarmuka grafis (GUI) atau berbasis web agar lebih mudah digunakan. Sistem juga dapat diintegrasikan dengan database untuk pengelolaan data yang lebih besar dan aman. Selain itu, algoritma optimasi rute seperti shortest path pada graph dapat diterapkan untuk meningkatkan efisiensi distribusi pangan di wilayah perkotaan. Penambahan fitur monitoring stok dan pelacakan distribusi secara real-time juga dapat meningkatkan kualitas sistem.

BAB VIII LAMPIRAN

8.1 Pembagian Tugas

KETERANGAN	NO	NAMA
Pembuatan Struktur Akun Github	I	Bama Putra Widiyanto
	II	Ahmad Niro Yogara
Pembuatan Power Point	I	Farhan Fajari Fadhlurrahman
	II	Deco Ardiandra
	III	Bama Putra Widiyanto
Pembuatan Code Program	I	Muhammad Surya Aldo Pratama
	II	Deco Ardiandra
	III	Ahmad Niro Yogara
	IV	Farhan Fajari Fadhlurrahman
	V	Bama Putra Widiyanto
Menjawab Soal Analisis	I	Farhan Fajari Fadhlurrahman
	II	Ahmad Niro Yogara
Pembuatan Laporan	BAB 1-2	Farhan Fajari Fadhlurrahman
		Muhannad Surya Aldo Pratama
	BAB 3-4	Deco Ardiandra
		Ahmad Niro Yogara
	BAB 5-6	Bama Putra Widiyanto
		Muhammad Surya Aldo Pratama
	BAB 7-8	Farhan Fajari Fadhlurrahman
		Deco Ardiandra

8.2 Tabel RunLengkap



8.2.1 Pertemuan Pertama 8 Mei 2026

Nama Anggota	Tugas	Durasi	Status
Farhan Fajari Fadhlurrahman	Menjawab Soal Analisi	5 Jam	Belum Selesai
Bama Putra Widiyanto	Pembuatan Akun github	5 Jam	Belum Selesai
Muhammad Surya Aldo Pratama	Pembuatan Promp Code	5 Jam	Revisi
Ahmad Niro Yogara	Pembuatan Akun Github	5 Jam	Belum Selesai
Deco Ardiandra	Pembuatan Promp Code	5 Jam	Revisi

8.2.2 Pertemuan kedua 14 Mei 2026

Nama Anggota	Tugas	Durasi	Status
Farhan Fajari Fadhlurrahman	1. Menjawab Soal Analisi 2. Pembuatan Laporan 3. Pembuatan Laporan	6 Jam	Done
Bama Putra Widiyanto	1. Pembuatan Akun Github 2. Pembuatan PPT 3. Pembuatan Laporan	6 Jam	Done
Muhammad Surya Aldo Pratama	1. Pembuatan Promp Code 2. Pembuatan Laporan 3. Menjawab Soal Anlisis	6 Jam	Done
Ahmad Niro Yogara	1. Pembuatan Akun Github 2. Pembuatan PPT 3. Pembuatan Laporan	6 Jam	Done
Deco Ardiandra	1. Pembuatan Promp Code 2. Pembuatan Laporan 3. Pembuatan PPT	6 Jam	Done